Doc. no.:	P1157R0
Date:	2018-07-14
Audience:	EWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

Multi-argument *constrained-parameter*

"[…] the multi-argument concept name introducer notion is the only fully general principled notation." [1]

Introduction

constrained upon introduced	**unconstrained upon introduced**

`template<EqualityComparable T>`
`bool operator==(list<T>, list<T>);`
	`template<class T>`
	`requires EqualityComparable<T>`
	`bool operator==(list<T>, list<T>);`

`template<EqualityComparableWith T U>`
`bool operator==(list<T>, list<U>);`
	`template<class T, class U>`
	`requires EqualityComparableWith<T, U>`
	`bool operator==(list<T>, list<U>);`

Motivation

The motivation of this proposal is to reduce the occurrence of unconstrained type parameters in generic library specifications. Take the following signature for example,

```
template<class T, class U>
requires EqualityComparableWith<T, U>
bool operator==(list<T> const& x, list<U> const& y);
```

Here we introduced two type parameters unconstrained to constrain them later. An alternative may introduce one:

```
template<class T, EqualityComparableWith<T> U>
bool operator==(list<T> const& x, list<U> const& y);
```

But if the designated concept takes only one argument, with *constrained-parameter* we don't have to have any unconstrained parameter:

```
template<EqualityComparable T>
bool operator==(list<T> const& x, list<T> const& y);
```

The author believes that it is worth to extend this functionality to apply to the concepts that take more than one arguments:

```
template<EqualityComparableWith T U>
bool operator==(list<T> const& x, list<U> const& y);
```

One might consider this to be a minor issue by claiming that multi-parameter type concepts are "rare" or "complex[2]." The rest of this section shows that these are misconceptions.

Multi-parameter type concepts are not uncommon

A survey on the future concepts library[3] and ranges library[4] shows that 47% percent of the concepts specified in the next standard can take multiple arguments:

	takes only one argument	may take multiple arguments
Standard Library Concepts	Integral SignedIntegral UnsignedIntegral Swappable Destructible DefaultConstructible MoveConstructible CopyConstructible Boolean EqualityComparable StrictTotallyOrdered Movable Copyable Semiregular Regular	Same DerivedFrom ConvertibleTo CommonReference Common Assignable SwappableWith Constructible EqualityComparableWith StrictTotallyOrderedWith Invocable RegularInvocable Predicate Relation StrictWeakOrder
The One Ranges Proposal	Readable WeaklyIncrementable Incrementable Iterator InputIterator OutputIterator ForwardIterator BidirectionalIterator RandomAccessIterator ContiguousIterator Permutable Range SizedRange View CommonRange InputRange ForwardRange BidirectionalRange RandomAccessRange ContiguousRange ViewableRange	Writable Sentinel SizedSentinel IndirectUnaryInvocable IndirectRegularUnaryInvocable IndirectUnaryPredicate IndirectRelation IndirectStrictWeakOrder IndirectlyMovable IndirectlyMovableStorable IndirectlyCopyable IndirectlyCopyableStorable IndirectlySwappable IndirectlyComparable Mergeable Sortable OutputRange

One might argue that “can take multiple arguments” does not necessarily mean that this concept wants to declare multiple type parameters at the same time. For example, `Sentinel` has only been used for declaring one type parameter in the ranges library:

```
template<Iterator I, Sentinel<I> S>
void advance(I& i, S bound);
```

However, `Sentinel<S, I>` already requires `I` to satisfy `Iterator`, so given the proposed functionality, we can rewrite the above generic function as

```
template<Sentinel S I>
void advance(I& i, S bound);
```

Although this version introduces the template parameters in the reversed order and has vague meaning, it implies that as long as a concept can take multiple arguments, it can potentially make use of the capability of introducing those template parameters at once. We can trivially resolve both issues mentioned earlier with a new concept that switches the parameters:

```
template<class I, class S>
concept Traversable = Sentinel<S, I>;

template<Traversable I S>
void advance(I& i, S bound);
```

Multi-parameter concepts are like multi-parameter functions

It is unclear what does “complex” mean in the term “complex concepts.” It may mean “more stuff to specify,” but there is no evidence showing that the multi-parameter concepts take more LoC to specify comparing to the unary concepts, especially when the `Boolean` concept takes 17 LoC. Or it may mean “more logical components,” then one might be surprised that `MoveConstructible` is not even a special case of `Constructible`,

```
template<class T>
concept MoveConstructible = Constructible<T, T> && ConvertibleTo<T, T>;
```

nor does `CopyConstructible`:

```
template<class T>
concept CopyConstructible = MoveConstructible<T> &&
    Constructible<T, T&> && ConvertibleTo<T&, T> &&
    Constructible<T, const T&> && ConvertibleTo<const T&, T> &&
    Constructible<T, const T> && ConvertibleTo<const T, T>;
```

We must recognize that multi-parameter concepts are not more special comparing to the multi-parameter functions. If they look complicated, the survey on the ranges library shows a possibility – given the status quo, the library can only introduce the template parameters one at a time. Here is an example: we almost always (32 out of 34) use the `IndirectUnaryPredicate` concept together with the `projected` wrapper,

```
template<InputIterator I, Sentinel<I> S, class Proj = identity,
         IndirectUnaryPredicate<projected<I, Proj>> Pred>
bool all_of(I first, S last, Pred pred, Proj proj = Proj{});
```

but defining a concept that captures this pattern (see also `IndirectlyComparable`)

```
template<class Proj, class F, class I>
concept ProjectedUnaryPredicate = IndirectUnaryPredicate<F, projected<I, Proj>>;
```

is not making the specification significantly more clear because we at least need to introduce one type parameter unconstrained:

```
template<InputIterator I, Sentinel<I> S, class Proj,
         ProjectedUnaryPredicate<I, Proj> Pred>
bool all_of(I first, S last, Pred pred, Proj proj);
```

This verbosity goes away as soon as we can declare multiple template parameters at once:

```
template<InputIterator I, Sentinel<I> S,
         ProjectedUnaryPredicate<I> Proj Pred>
bool all_of(I first, S last, Pred pred, Proj proj);
```

Combining with

```
template<class I, class S>
concept InputTraversable = InputIterator<I> && Sentinel<S, I>;
```

, we can rewrite the full declarations of `ranges::all_of`

```
template<InputIterator I, Sentinel<I> S, class Proj = identity,
    IndirectUnaryPredicate<projected<I, Proj>> Pred>
  bool all_of(I first, S last, Pred pred, Proj proj = Proj{});
template<InputRange Rng, class Proj = identity,
    IndirectUnaryPredicate<projected<iterator_t<Rng>, Proj>> Pred>
  bool all_of(Rng&& rng, Pred pred, Proj proj = Proj{});
```

into

```
template<InputTraversable I S, IndirectUnaryPredicate<I> Pred>
  bool all_of(I first, S last, Pred pred);
template<InputTraversable I S, ProjectedUnaryPredicate<I> Proj Pred>
  bool all_of(I first, S last, Pred pred, Proj);

template<InputRange Rng, IndirectUnaryPredicate<iterator_t<Rng>> Pred>
  bool all_of(Rng&& rng, Pred pred);
template<InputRange Rng, ProjectedUnaryPredicate<iterator_t<Rng>> Proj Pred>
  bool all_of(Rng&& rng, Pred pred, Proj);
```

Now all the template parameters are constrained at the time of being introduced. There is little need for mental type arithmetic after only a few tweaks to the complex specifications that would not benefit from the proposed functionality at first glance, what you see then is what you get.

Discussion

Why such an alien syntax?

Because the idea is alien. C++ yet has the following setting: introducing both a and b is the only way for X to declare anything. For example, let X be `int`, then `int a, b;` declares `a` and `b`, but `int` can also introduce only one name with `int a;`. Given `pair<bool, int> foo();`, `auto [a, b] = foo();` declares `a` and `b`; but `auto` can also introduce only one name with `auto a = foo();`. The proposed syntax, *qualified-concept-name* followed by a list of identifiers that has no separator token between them, ensures sufficient unfamiliarity so that nothing else in the language (such as *enum-specifier*) come to mind when a user see this in code.

About why this specific syntax, continuing our “function” metaphor: a unary function is a multi-parameter function with an arity of 1, so the author comes up with a design such that a unary *constrained-parameter*, as specified in the working paper[5], not only semantically, but also syntactically, is a multi-argument constrained parameter with an arity of 1.

Template declaration is ugly!

So we should make *template-declaration* less ugly. Here is a proposal:

> *template-head*:
>
> `template` $_{opt}$ `<` *template-parameter-list* `>` *requires-clause*$_{opt}$

> *explicit-specialization*:
>
> `template` $_{opt}$ `< >` *declaration*

With this change, a generic function declaration will get closer to a generic lambda:

```
auto f = <Iterator I>(I first, I last) { /* ... */ };
  <Iterator I> auto g(I first, I last) { /* ... */ }

auto f = <Traversable I S>(I first, S last) { /* ... */ };
  <Traversable I S> auto g(I first, S last) { /* ... */ }
```

Should we reserve this syntax for adjective conjunction concepts?

It was suggested that a constrained parameter such as `Sortable Container X` should declare `X` that requires `Sortable<X> && Container<X>`. There are two premises for this interpretation to be useful:

1. Both concepts involved are unary;
2. One concept does not subsume the other.

Whether it is possible for all `Containers` being `Sortable` or all `Sortable` being `Containers` is something we cannot tell from undefined concepts. Therefore the author surveyed the ranges library to look for the signatures that have constraints satisfying 1) and 2).

The survey shows that among all the constraints additionally supplied, one concept (`Permutable`) involved is unary; among the 8 algorithms (16 signatures) where `Permutable` is spelled out, 3 signatures (iterator version `reverse`, `stable_partition`, `shuffle`) can actually benefit from the adjective conjunction syntax. Note that `Permutable` already subsumes `ForwardIterator` so sometimes the later is not necessary, but the wording has the convention of not to use an adjective *concept-name* to declare a *constrained-parameter*.

So it seems that the suggested idea is not very useful within one generic library. What about the case when the concepts come from different generic libraries? The author does not have real code to make a call but suspects that such a syntax may be misleading because partial ordering independently defined concepts can give unexpected results.

To answer the question in the heading: not worth it.

How do I omit the template parameter names?

The short answer is you cannot. A more professional answer is that you can only omit the *prototype parameter* for a given concept. This rule explains the status quo and uniformly applies to both unary concepts and multi-parameter concepts. For example,

```
template<class T>
  concept EqualityComparable = /* ... */;
template<class T, class U>
  concept EqualityComparableWith = /* ... */;
```

The *constrained-parameters* `EqualityComparable X`, `EqualityComparableWith X Y`, and `EqualityComparableWith<Y> X` are legal because they are not omitting any name; `EqualityComparable` and `EqualityComparableWith<Y>` are also legal because they are omitting their prototype parameter `T`. But `EqualityComparableWith X` is illegal because it intends to omit `U` which is not a prototype parameter.

The outcome of the design is WYSIWYG – a reader of the code can easily infer how many template parameters there are from a declaration. If a *qualified-concept-name* has no identifier to follow it, it introduces an unnamed template parameter; otherwise, the number of the identifiers following it is the number of template parameters it declares.

Where are the default arguments?

They are not sufficiently motivated given the examples. It may also be worth noticing that neither Concepts TS[6] nor the in-place syntax[7] allows default arguments on the multi-argument concept name introducer notions.

Technical Specification

> *constrained-parameter*:
>
> *qualified-concept-name … identifier*$_{opt}$
>
> *qualified-concept-name identifier*$_{opt}$ *default-template-argument*$_{opt}$
>
> *qualified-concept-name identifier-seq*
>
> *identifier-seq*:
>
> *identifier*
>
> *identifier-seq identifier*
>
> A *constrained-parameter* introduces a *constraint-expression* (12.4.2 (http://eel.is/c++draft/temp.constr.decl)). The expression is derived from the *qualified-concept-name* Q in the *constrained-parameter*, its designated concept C, and the declared template parameters P_1, P_2, …, P_n.
>
> An *id-expression* E is formed as follows. If Q is a *concept-name*, then E is C<P_1, P_2, …, P_n>. Otherwise, Q is a *partial-concept-id* of the form C<A_1, A_2, …, A_m>, and E is C<P_1, P_2, …, P_n, A_1, A_2, …, A_m>.
>
> E is the introduced *constraint-expression*.

This paper proposes only a syntax translation. The capability of this syntax is up to the capability of a unary *constrained-parameter*. If YAACD[8], or a part of it, is adopted, a multi-argument *constrained-parameter* will introduce only type parameters; otherwise, given the status quo, a multi-argument *constrained-parameter* will be able to introduce template parameters of mixed kinds (type, non-type, template):

```
template<class T, auto N>
concept Array = is_array_v<T[N]>;

template<Array T N>
void foo(std::array<T, N>&);
```

References

1. Stroustrup, Bjarne. P1079 *A minimal solution to the concepts syntax problems*. http://open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1079r0.pdf (http://open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1079r0.pdf) ↩

2. Köppe, Thomas. P1142R0 *Thoughts on a conservative terse syntax for constraints*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1142r0.html (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1142r0.html) ↩

3. Carter, Casey, and Eric Niebler. P0898R3 *Standard Library Concepts*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0898r3.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0898r3.pdf) ↩

4. Niebler, Eric, et al. P0896R2 *The One Ranges Proposal*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0896r2.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0896r2.pdf) ↩

5. Smith, Richard. N4762 *Working Draft, Standard for Programming Language C++*. https://github.com/cplusplus/draft/raw/master/papers/n4762.pdf (https://github.com/cplusplus/draft/raw/master/papers/n4762.pdf) ↩

6. Sutton, Andrew. N4674 *Working Draft, C++ extensions for Concepts*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4674.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4674.pdf) ↩

7. Sutter, Herb. P0745R1 *Concepts in-place syntax*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0745r1.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0745r1.pdf) ↩

8. Voutilainen, Ville, et al. P1141R0 *Yet another approach for constrained declarations*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1141r0.html (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1141r0.html) ↩